

Estratégias Avançadas de Ofuscação e Evasão de Detecção em Ambientes Android Containerizados: Uma Análise Técnica Profunda do Redroid no Linux

1. Introdução e Contextualização Arquitetural

A virtualização de sistemas operacionais móveis transcendeu o desenvolvimento de software tradicional, tornando-se um componente crítico em pesquisas de segurança, automação de testes, análise de malware e, crescentemente, na execução de aplicações de alta performance em nuvem (*Cloud Gaming*). Dentro deste espectro, o **Redroid (Remote Android)** emergiu como uma solução disruptiva. Diferente dos emuladores tradicionais baseados em QEMU (como o Android Virtual Device - AVD ou Genymotion), que emulam uma arquitetura de hardware completa sobre um hypervisor, o Redroid opera como um container Docker, compartilhando o kernel do sistema operacional host (Linux) com o espaço do usuário Android.¹

Esta distinção arquitetural — containerização versus virtualização completa — oferece benefícios substanciais de desempenho, permitindo densidade de instâncias muito superior e latência reduzida devido à eliminação da sobrecarga de tradução de instruções de hardware. No entanto, essa mesma arquitetura expõe vetores de detecção únicos e profundos. Enquanto emuladores tradicionais lutam para esconder dispositivos de hardware virtualizados (como o "Goldfish" da Google), o Redroid luta para esconder a própria natureza do kernel Linux subjacente, que frequentemente revela ser um kernel de servidor (Ubuntu, Debian, Fedora) e não um kernel Android modificado.¹

O presente relatório técnico dissectiona, com exaustividade, as metodologias para mascarar a presença do Redroid. A análise abrange desde a manipulação de baixo nível de chamadas de sistema e drivers de kernel até a injeção de código em tempo de execução via *Zygisk*, culminando na complexa interação com a *Google Play Integrity API*. O objetivo é fornecer um roteiro técnico para pesquisadores e engenheiros que necessitam de um ambiente Android containerizado indistinguível de um dispositivo físico legítimo.

2. Fundamentos da Detecção em Ambientes Containerizados

Para construir uma defesa eficaz (mascaramento), é imperativo compreender a ofensiva (detecção). Os mecanismos de detecção modernos, empregados por soluções como

Appdome, *Byfron* (Roblox), *Momo* e aplicações bancárias, utilizam uma abordagem em camadas para identificar anomalias no ambiente de execução.

2.1. O Paradoxo do Kernel Compartilhado

A vulnerabilidade mais crítica do Redroid reside no compartilhamento do kernel. Em um dispositivo Android físico, o kernel é compilado com configurações específicas (ex: `CONFIG_ANDROID_BINDER_IPC`, `CONFIG_ASHMEM`) e possui uma string de versão característica (ex: `4.19.157-android12-9-g...`).

No Redroid, o ambiente Android herda o kernel do host. Se o host for um servidor Ubuntu 22.04 rodando o kernel 5.15 genérico, qualquer aplicação dentro do container que execute a `syscall uname()` ou leia `/proc/version` receberá a string do host: `Linux localhost 5.15.0-91-generic #101-Ubuntu SMP...`¹

Esta string é uma "assinatura" imediata de um ambiente não nativo. Aplicações de segurança verificam isso antes mesmo de procurar por binários su ou aplicativos suspeitos. Além disso, a ausência de certos patches específicos do Android no kernel mainline do Linux pode causar discrepâncias comportamentais sutis em threads e gerenciamento de memória, que heurísticas avançadas podem detectar.²

2.2. Subsistema Binder e Alocação de Memória

O Android depende pesadamente do driver **Binder** para Comunicação Interprocessual (IPC) e do **Ashmem** (Android Shared Memory) para gerenciamento de memória.

- **BinderFS vs. Binder Devices:** Kernels Linux modernos (5.0+) movem-se em direção ao `binderfs`, um sistema de arquivos para gerenciar dinamicamente dispositivos binder. Dispositivos Android legados esperam dispositivos de caractere estáticos em `/dev/binder`, `/dev/hwbinder` e `/dev/vndbinder`. O Redroid tenta gerenciar isso, mas a forma como esses dispositivos são montados e apresentados pode divergir de um dispositivo real, criando um vetor de detecção.⁵
- **Memfd vs. Ashmem:** O driver `ashmem` foi depreciado em favor do `memfd` (Memory File Descriptor) padrão do Linux. Embora o Redroid suporte `androidboot.use_memfd=1`, aplicações antigas ou verificações de integridade podem procurar especificamente pelo driver `ashmem`, cuja ausência ou emulação imperfeita sinaliza um ambiente virtualizado.⁸

2.3. Assinaturas de Propriedades e Arquivos

O sistema de propriedades do Android (`build.prop`) é a primeira linha de interrogação para apps curiosos.

- **Tags de Build:** Imagens de emulador geralmente possuem `ro.build.tags=test-keys`, indicando uma imagem de desenvolvimento assinada com chaves públicas genéricas, em oposição a `release-keys` de dispositivos de produção.

- **Fingerprint do Dispositivo:** A string `ro.build.fingerprint` combina marca, modelo, versão e build ID. Discrepâncias entre esta string e as capacidades reais do hardware (ex: um fingerprint de Pixel 6 reportando uma GPU Intel ou gráficos de software llvmpipe) são fatais para a credibilidade do ambiente.¹⁰
- **Artefatos de Arquivos:** A presença de binários como `su`, `daemonsu`, ou pastas como `/system/xbin/mu` são verificações triviais, porém eficazes, se o acesso root não for devidamente ocultado.¹²

3. Infraestrutura de Ocultação: Configuração do Host e Imagem

A base de um mascaramento bem-sucedido começa fora do Android, no próprio sistema operacional host e na construção da imagem Docker.

3.1. Preparação do Kernel do Host Linux

Para mitigar a detecção baseada em kernel, a abordagem ideal envolve a compilação de um kernel customizado para o host ou o uso de módulos de kernel carregáveis (LKM) que suportem as funcionalidades nativas do Android.

Requisitos Essenciais:

1. **Módulos Binder e Ashmem:** É crucial garantir que `binder_linux` e `ashmem_linux` estejam carregados. Em distribuições como Ubuntu, o pacote `linux-modules-extra` fornece esses drivers.
 - *Comando de Carregamento:* `modprobe binder_linux devices="binder,hwbinder,vndbinder"`.⁹
 - *Análise Crítica:* A ordem e os nomes dos dispositivos são importantes. Alguns kernels modernos ("Zen Kernel" no Arch Linux) usam `binderfs` por padrão, o que pode quebrar a inicialização do Redroid se a imagem esperar nós estáticos em `/dev`. A solução envolve montar o `binderfs` corretamente ou usar um kernel que permita a criação legada desses nós.⁵

3.2. Personalização da Imagem Redroid (Redroid-Script)

Utilizar a imagem padrão do Docker Hub (`redroid/redroid:11.0.0-latest`) é insuficiente para evasão, pois ela carece de componentes essenciais como GApps (Google Apps) e Magisk, e contém binários de depuração. A ferramenta **redroid-script**¹⁵ é o padrão da indústria para "cozinhar" uma imagem robusta.

Componentes Críticos na Construção:

- **Magisk (Root Gerenciado):** Ao contrário do binário `su` simples incluído em builds de depuração do AOSP, o Magisk oferece "systemless root" (root sem modificar a partição

do sistema de forma destrutiva) e, crucialmente, uma infraestrutura de módulos para ocultação (Zygisk). O script injeta o Magisk na partição de boot ou initramfs do container.¹⁷

- **Libndk vs. Libhoudini (Tradução ARM):** A maioria dos jogos e apps de alta segurança são compilados para arquitetura ARM64. Como servidores Redroid rodam tipicamente em x86_64 (AMD/Intel), uma camada de tradução é obrigatória.
 - **Libndk:** Derivado do emulador oficial do Android Studio. Geralmente oferece melhor desempenho em CPUs AMD e é menos propenso a falhas gráficas em certos jogos.¹⁵
 - **Libhoudini:** Tecnologia da Intel. Preferível em hosts com CPUs Intel.
 - *Implicação de Detecção:* A presença dessas bibliotecas (libndk_translation.so) é detectável. No entanto, elas são essenciais. A estratégia de mascaramento foca em esconder que essas bibliotecas estão sendo usadas para tradução, ou simular que o dispositivo é nativamente ARM (embora isso seja difícil sem emulação completa lenta).¹⁹

Comando de Construção Otimizado:

Bash

```
python redroid.py \  
-a 11.0.0 \      # Versão Android (11 é o "sweet spot" de estabilidade/compatibilidade)  
-c docker \     # Container engine  
-m \            # Magisk (Essencial para Zygisk/Shamiko)  
-g \            # GApps (Necessário para Play Services e Play Integrity)  
-n \            # Libndk (Tradução ARM)  
-w              # Widevine (DRM para apps de streaming)
```

Este processo gera uma imagem local (ex:

redroid/redroid:11.0.0-magisk-gapps-ndk-widevine) que servirá como base para as camadas de ofuscação subsequentes.¹⁵

4. Camada de Evasão 1: Gestão de Root e Zygisk

A mera presença de acesso administrativo (root) é o principal gatilho para bloqueios de segurança. O objetivo é ter root para modificar o sistema, mas esconder esse fato de todos os outros aplicativos.

4.1. Magisk em Ambientes Containerizados

No Android tradicional, o Magisk modifica a partição boot.img para sequestrar o processo de

inicialização. No Redroid, como não há um "bootloader" tradicional, a integração é feita modificando os scripts de inicialização `init.rc` dentro do container para lançar o daemon do Magisk (`magiskd`).¹³

Problemas de Detecção do Magisk:

- **Nome do Pacote:** O aplicativo gerenciador do Magisk tem, por padrão, o pacote `com.topjohnwu.magisk`. Apps buscam por essa string.
- **Binários:** A presença de `su` em diretórios do PATH (`/sbin`, `/system/bin`).
- **Serviços:** Daemons rodando em background.

4.2. A Revolução Zygisk (Zygote + Magisk)

O **Zygisk** permite que o Magisk execute código no contexto de cada aplicativo Android. O processo Zygote é o pai de todos os aplicativos Android; ao injetar código nele, o Zygisk pode "limpar" o ambiente de execução de um aplicativo específico antes mesmo que ele execute sua função `main`.²¹

Implementação no Redroid:

1. Após iniciar o container, o Zygisk deve ser ativado nas configurações do Magisk App.
2. É necessário reiniciar o container (`docker restart`) para que a injeção no Zygote ocorra.
3. *Conflito Crítico:* Em algumas versões do Redroid, ativar o Zygisk quebra a tradução nativa (`libndk`) para apps 32-bit. A solução envolve forçar o uso de imagens 64-bit-only ou aplicar patches específicos no `ro.dalvik.vm.native.bridge`.¹⁵

4.3. Shamiko: Ocultação de Última Geração

O método antigo "Magisk Hide" foi removido. A "DenyList" (Lista de Negação) nativa do Magisk é insuficiente pois apenas remove os *hooks* do Magisk, não esconde os artefatos residuais. O módulo **Shamiko** resolve isso.²¹

Mecanismo de Ação:

- O Shamiko lê a configuração da DenyList do Magisk.
- *Configuração Contraintuitiva:* O usuário deve configurar os apps na DenyList (Google Play Services, Apps Bancários), mas **NÃO** deve ativar a opção "Enforce DenyList" no Magisk. O Shamiko detecta essa configuração e assume o controle, ocultando não apenas o root, mas também a própria injeção do Zygisk, isolando o processo do aplicativo alvo.²¹
- Isso permite que módulos como LSPosed continuem funcionando (se não estiverem na lista de bloqueio), enquanto o root permanece invisível para os alvos.

5. Camada de Evasão 2: Integridade da Play API e

Identidade

A **Google Play Integrity API** (sucessora do SafetyNet) é o árbitro final de confiança. Ela avalia se o pedido vem de um binário de app genuíno, instalado pelo Google Play, rodando em um dispositivo Android genuíno.

5.1. Os Três Níveis de Integridade

1. **MEETS_BASIC_INTEGRITY:** Verifica se o ambiente não é um emulador conhecido e não tem root óbvio. O Redroid padrão falha aqui.
2. **MEETS_DEVICE_INTEGRITY:** Verifica se o software do dispositivo corresponde a um dispositivo Android certificado pelo Google. Exige um *fingerprint* válido e consistente.
3. **MEETS_STRONG_INTEGRITY:** A barreira mais alta. Exige prova criptográfica gerada dentro de um **TEE (Trusted Execution Environment)** de hardware (ex: TrustZone, Titan M). Como o Redroid roda em hardware de servidor genérico (x86), ele não possui esse hardware seguro específico.²⁵

5.2. Superando as Barreiras: Play Integrity Fix (PIF)

Para passar nas verificações BASIC e DEVICE, utiliza-se o módulo **Play Integrity Fix** (por *chiteroman* ou *osmOsis*).

- **Estratégia do Fingerprint Banido:** O módulo injeta propriedades de sistema (build.prop) de dispositivos reais antigos ou menos populares que não impõem a verificação de hardware forte.
- **O Jogo de Gato e Rato:** O Google bane constantemente fingerprints usados publicamente. Usuários avançados de Redroid devem extrair seus próprios fingerprints de dispositivos físicos reais usando ferramentas como o *Termux* e scripts de extração de build.prop, injetando-os no arquivo pif.json do módulo.²⁷

5.3. A Busca pelo Graal: Strong Integrity e TrickyStore

Passar na verificação STRONG em um emulador é teoricamente impossível sem hardware dedicado. No entanto, pesquisadores desenvolveram o **TrickyStore**.

- **Keybox XML:** O TrickyStore utiliza um arquivo chamado keybox.xml, que contém chaves privadas raiz de TEEs de dispositivos OEM comprometidos ou vazados.
- **Simulação de TEE:** O módulo intercepta as chamadas da API de Keystore do Android e usa o keybox.xml para assinar digitalmente os desafios de atestação. Se o certificado no keybox.xml não tiver sido revogado pelo Google, o servidor acreditará que o Redroid é, de fato, o dispositivo físico ao qual aquelas chaves pertenciam.²⁹
- **Implementação:** O arquivo keybox.xml deve ser colocado em /data/adb/tricky_store/ com permissões restritas (600 ou 644). Encontrar um Keybox válido é extremamente difícil e eles são "queimados" rapidamente quando compartilhados publicamente.³¹

6. Camada de Evasão 3: Spoofing de Hardware e Kernel

Quando a análise de software falha, os sistemas de detecção olham para o hardware e o kernel.

6.1. Ocultação de Informações do Kernel (uname Spoofing)

Conforme identificado na seção 2.1, o comando `uname -a` revela o kernel do host. Para mascarar isso em um container Redroid, técnicas de user-space são limitadas.

- **Abordagem via `seccomp` / `ptrace`:** É possível interceptar a syscall `uname` e reescrever a estrutura de retorno com dados falsos. No entanto, o uso de `ptrace` reduz drasticamente o desempenho e é facilmente detectável (o próprio fato de o processo estar sendo "traced" é suspeito).³³
- **KernelSU e suSFS:** A solução de ponta atual é o uso do **KernelSU** (uma alternativa ao Magisk que opera inteiramente no espaço do kernel) combinado com o **suSFS** (Systemless Super File System).
 - **Mecanismo:** O suSFS pode fazer spoofing de funções do kernel diretamente na memória, alterando o que é retornado por `/proc/version` e `uname` apenas para processos específicos (os apps maliciosos/alvos), enquanto mantém a verdade para o sistema operacional.⁴
 - **Desafio no Redroid:** Para usar o KernelSU/suSFS no Redroid, o kernel do **host Linux** deve ser compilado com os patches do KernelSU. Isso não é uma instalação de aplicativo; é uma alteração na infraestrutura do servidor. Se você controla o host, esta é a forma mais robusta de ocultação existente.³⁶

6.2. Emulação de Sensores e Periféricos

Um dispositivo imóvel com 100% de bateria constante e conectado via Ethernet é suspeito.

- **Fake Wifi (LSPosed):** Módulos como "Fake My Connection" ou "Fake Wifi" interceptam chamadas ao `ConnectivityManager` e `WifiManager`. Eles reportam que o dispositivo está conectado a uma rede Wi-Fi (com SSID e BSSID forjados) em vez de Ethernet, o que é crucial para apps que bloqueiam downloads em dados móveis ou exigem Wi-Fi para funcionar.³⁸
- **Bateria e Térmica:** Scripts simples podem alterar `/sys/class/power_supply/battery/capacity` dentro do container, mas a alteração é estática. Soluções mais avançadas envolvem daemons que variam randomicamente o nível de bateria e a temperatura da CPU para simular uso real, evitando detecções heurísticas baseadas em telemetria estática.⁸

7. Guia de Implementação Prática: Construindo a

"Golden Image"

Abaixo, consolida-se a pesquisa em um procedimento passo-a-passo para implantar um ambiente Redroid de alta furtividade.

7.1. Configuração do Docker Run

A invocação do container deve definir propriedades de sistema iniciais que dificultem a identificação antes mesmo do boot.

Bash

```
docker run -itd \
  --privileged \
  --name redroid_stealth \
  -v ~/data:/data \
  -p 5555:5555 \
  redroid/redroid:11.0.0-magisk-gapps-ndk \
  \
  # Configurações de Hardware Falso
  androidboot.hardware=qcom \
  ro.hardware=qcom \
  ro.product.name=oriole \
  ro.product.model="Pixel 6" \
  ro.product.manufacturer=Google \
  ro.product.device=oriole \
  ro.build.product=oriole \
  ro.build.id=SP2A.220305.013.A3 \
  \
  # Desativação de Flags de Debug
  ro.secure=1 \
  ro.debuggable=0 \
  ro.build.type=user \
  ro.build.tags=release-keys \
  \
  # Configuração Gráfica (GPU do Host)
  androidboot.redroid_gpu_mode=host \
  \
  # Fingerprint Completo (Deve coincidir com o modelo acima)
  ro.build.fingerprint="google/oriole/oriole:12/SP2A.220305.013.A3/8223566:user/release-keys"
```


Nota: A propriedade `ro.secure=1` é importante para simular um ambiente de produção ("User Build"), mas pode interferir na inicialização inicial do daemon do Magisk em algumas versões. Recomenda-se iniciar com `ro.secure=0`, configurar o Magisk, e depois alterar para 1 via módulo "MagiskHide Props Config".¹¹

7.2. Configuração Pós-Instalação (In-Container)

1. **Instalação do App Magisk:** Instalar o Magisk.apk (renomeado do stub) para gerenciar o root.
2. **Configuração Zygisk:** Ativar Zygisk nas configurações e **reiniciar o container**.
3. **Instalação de Módulos (Ordem Recomendada):**
 - *LSPosed (Zygisk Release):* Framework base para hooking.
 - *Play Integrity Fix (Versão mais recente):* Para passar em Basic/Device Integrity.
 - *Shamiko:* Instalar e garantir que "Enforce DenyList" esteja **DESATIVADO**.
 - *Hide My Applist:* Configurar para ocultar pacotes de root (com.topjohnwu.magisk, org.lsposed.manager)³⁵ de apps alvo.
4. **Configuração da DenyList:** No Magisk, vá em "Configure DenyList" e marque:
 - Google Play Services (com.google.android.gms) - Especialmente o processo com.google.android.gms.unstable.
 - Google Play Store (com.android.vending).
 - App Alvo (ex: App Bancário, Jogo).
5. **Validação:** Utilize apps como YASNAC (Yet Another SafetyNet Attestation Checker) ou *Play Integrity API Checker* para validar os resultados. Se o resultado for "MEETS_DEVICE_INTEGRITY", o mascaramento básico foi bem-sucedido.

8. Solução de Problemas e Limitações Conhecidas

8.1. Detecção "Momo" e Bootloops

O app de detecção *Momo* (Magisk Detection) é a referência em encontrar falhas no mascaramento. Se o Momo detectar "Zygote injected", significa que o Shamiko não está funcionando ou o Zygisk foi detectado heurísticamente.

- **Solução:** Verificar se há módulos conflitantes. Módulos que modificam propriedades do sistema globalmente podem disparar o Momo. Prefira modificações direcionadas via LSPosed.⁴²

8.2. Instabilidade Gráfica e GPU

Ocultar a GPU (`ro.hardware.egl`) pode causar falhas em jogos 3D. O Redroid usa renderização via GPU do host, o que expõe drivers como virtio-gpu ou drivers NVIDIA/Intel do host.

- **Análise:** Não há solução perfeita de software para esconder o renderizador GL sem matar a performance (usando renderização de software swiftshader). A melhor aposta é

spoofing de strings GL via módulos do GLTools ou similares, embora isso seja arriscado.¹⁴

9. Conclusão

O mascaramento eficaz de um emulador Redroid no Linux é um exercício de complexidade técnica que exige alinhamento vertical desde o kernel do host até a camada de aplicação Android. A combinação de **Redroid-Script** para construção da imagem, **Magisk/Zygisk/Shamiko** para controle de acesso e **Play Integrity Fix/TrickyStore** para atestação remota constitui o estado da arte atual.

No entanto, a barreira final — a detecção do kernel Linux compartilhado e a falta de hardware criptográfico genuíno — permanece um desafio para ambientes de alta segurança (Strong Integrity). A evolução futura aponta para a necessidade de kernels de host customizados (KernelSU/suSFS) e a integração de "Keyboxes" privadas para manter a ilusão de legitimidade frente a algoritmos de detecção cada vez mais invasivos.

Tabela Comparativa de Técnicas de Mascaramento

Vetor de Detecção	Técnica de Mitigação	Ferramenta/Módulo Principal	Eficácia	Complexidade
Binário SU / Root	Ocultação de Processo e Arquivo	Magisk + Shamiko	Alta	Média
Injeção de Zygote	Limpeza de Rastreamento de Memória	Shamiko	Alta	Baixa
Props do Sistema	Spoofing de build.prop	Play Integrity Fix / Docker Args	Média	Baixa
Integridade da API	Spoofing de Fingerprint/Keystore	Play Integrity Fix / TrickyStore	Alta (Device) / Média (Strong)	Alta
Kernel Version	Spoofing de Syscall / Kernel	KernelSU + suSFS	Alta	Extrema

	Patch	(Host-side)		
Conexão de Rede	Simulação de Wi-Fi sobre Ethernet	FakeWifi (LSPosed)	Alta	Média
Arquitetura CPU	Tradução ARM Transparente	Libndk + Ocultação de Libs	Média	Média

Referências Citadas no Texto

1.

Referências citadas

1. Hiding Redroid (Android Container) from Roblox Byfron/Appdome - Kernel & Root Detection Bypass? : r/Magisk - Reddit, acessado em janeiro 26, 2026, https://www.reddit.com/r/Magisk/comments/1qfedwu/hiding_redroid_android_container_from_roblox/
2. replace android kernel · Issue #503 · remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-doc/issues/503>
3. redroid-doc/deploy/README.md at master - GitHub, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-doc/blob/master/deploy/README.md>
4. sidex15/susfs4ksu-module: An addon root hiding service for KernelSU - GitHub, acessado em janeiro 26, 2026, <https://github.com/sidex15/susfs4ksu-module>
5. Support for the binderfs instead of binder · Issue #3 · remote-android/redroid-modules, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-modules/issues/3>
6. The Android binderfs Filesystem - The Linux Kernel documentation, acessado em janeiro 26, 2026, <https://docs.kernel.org/admin-guide/binderfs.html>
7. [Feature Request] Add support for modern kernels using binderfs · Issue #859 · remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-doc/issues/859>
8. Android Battery Mocking tutorial - Medium, acessado em janeiro 26, 2026, <https://medium.com/@kaushiknsanji/android-battery-mocking-tutorial-988927fa4a35>
9. remote-android/redroid-modules: redroid kernel modules (binderfs, ashmem etc.) - GitHub, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-modules>
10. How to change a device product properties (manufacturer, model, etc.) with

Genymotion SaaS?, acessado em janeiro 26, 2026,
<https://support.genymotion.com/hc/en-us/articles/115001338045-How-to-change-a-device-product-properties-manufacturer-model-etc-with-Genymotion-SaaS>

11. How to pass basic/device/strong Play Integrity checks using Magisk? · Issue #49 · ayasa520/redroid-script - GitHub, acessado em janeiro 26, 2026,
<https://github.com/ayasa520/redroid-script/issues/49>
12. Defeating Android Emulator Detection - Virtue Security, acessado em janeiro 26, 2026, <https://www.virtuesecurity.com/defeating-android-emulator-detection/>
13. Magisk support #207 - remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026, <https://github.com/remote-android/redroid-doc/issues/207>
14. Install ReDroid for playing Android games in Docker containers on Linux PC, with GPU acceleration · Ivon's Blog, acessado em janeiro 26, 2026,
<https://ivonblog.com/en-us/posts/redroid-android-docker/>
15. ayasa520/redroid-script - GitHub, acessado em janeiro 26, 2026,
<https://github.com/ayasa520/redroid-script>
16. rote66/redroid-script-openwrt - GitHub, acessado em janeiro 26, 2026,
<https://github.com/rote66/redroid-script-openwrt>
17. MagiskOnRedroid - GitHub Gist, acessado em janeiro 26, 2026,
<https://gist.github.com/assiless/a23fb52e8c6156db0474ee8973c4be66>
18. Magisk support · Issue #207 · remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026,
https://github.com/remote-android/redroid-doc/issues/207?timeline_page=1
19. Can't get ndk_translation to work on android 11 · Issue #18 · remote-android/redroid-doc, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/18>
20. Help with app compatibility - waydroid - Reddit, acessado em janeiro 26, 2026,
https://www.reddit.com/r/waydroid/comments/1l909b6/help_with_app_compatibility/
21. guide_hma/Magisk-Shamiko.md at master - GitHub, acessado em janeiro 26, 2026, https://github.com/mModule/guide_hma/blob/master/Magisk-Shamiko.md
22. Magisk Hide vs Zygisk & DenyList | Are they Same or It's Different? Android Root With ... - YouTube, acessado em janeiro 26, 2026,
<https://www.youtube.com/watch?v=rDNL6Qe7Gwc>
23. Magisk & Zygisk 2023: Hide Root Detection | Shamiko Lsposed - YouTube, acessado em janeiro 26, 2026,
<https://www.youtube.com/watch?v=juKRQWzC8qM>
24. Detecting Shamiko & Zygisk Root Hiding on Android (2025) — The Definitive Developer Guide | by Arnav singh | Medium, acessado em janeiro 26, 2026,
<https://medium.com/@arnavsinghinfosec/detecting-shamiko-zygisk-root-hiding-on-android-2025-the-definitive-developer-guide-71beac4a378d>
25. Overview of the Play Integrity API - Android Developers, acessado em janeiro 26, 2026, <https://developer.android.com/google/play/integrity/overview>
26. Returned integrity verdict format - Google Play - Android Developers, acessado em janeiro 26, 2026, <https://developer.android.com/google/play/integrity/verdicts>

27. Create Fingerprint/Build Prop JSON and Pass Play Integrity Test [Passing Values Given!], acessado em janeiro 26, 2026, https://www.youtube.com/watch?v=4Omyr_OR7eY
28. [Tutorial] Guide on fixing play integrity on rooted device. : r/Magisk - Reddit, acessado em janeiro 26, 2026, https://www.reddit.com/r/Magisk/comments/1js8qm3/tutorial_guide_on_fixing_play_integrity_on_rooted/
29. [Tutorial] Using TrickyStore with Zygisk Next for Integrity Bypass . [To minimize conflicts] : r/Magisk - Reddit, acessado em janeiro 26, 2026, https://www.reddit.com/r/Magisk/comments/1gegtr4/tutorial_using_trickystore_with_zygisk_next_for/
30. [Tutorial] Valid Keystore Fix on TrickyStore and passing PlayIntegrity : r/Magisk - Reddit, acessado em janeiro 26, 2026, https://www.reddit.com/r/Magisk/comments/1lbs4al/tutorial_valid_keystore_fix_on_trickystore_and/
31. New Unrevoked Keybox File [25th] and Keybox Module [v1.5] are Here! - YouTube, acessado em janeiro 26, 2026, https://www.youtube.com/watch?v=nkRb_Ou-9VU
32. Where / how to find keybox.xml files for TrickyStore? : r/androidroot - Reddit, acessado em janeiro 26, 2026, https://www.reddit.com/r/androidroot/comments/1fwn3wc/where_how_to_find_keyboxxml_files_for_trickystore/
33. UNIX ptrace() block child's system calls - linux - Stack Overflow, acessado em janeiro 26, 2026, <https://stackoverflow.com/questions/4705070/unix-ptrace-block-childs-system-calls>
34. SYS_Ptrace Capability for Linux/AMD64 - Docker Community Forums, acessado em janeiro 26, 2026, <https://forums.docker.com/t/sys-ptrace-capability-for-linux-amd64/138482>
35. This guide is for all Android devices to hide properly all root detections and successfully run banking apps as expected - GitHub Gist, acessado em janeiro 26, 2026, <https://gist.github.com/TheUnrealZaka/042040a1700ad869d54e781507a9ba4f>
36. How to Spoof/Fake Custom Kernel to Stock Kernel via SUSFS - YouTube, acessado em janeiro 26, 2026, <https://www.youtube.com/watch?v=TJTJx1loS2w>
37. What are actual solutions to the anti cheat issue? : r/linux_gaming - Reddit, acessado em janeiro 26, 2026, https://www.reddit.com/r/linux_gaming/comments/1mg9v7g/what_are_actual_solutions_to_the_anti_cheat_issue/
38. Fake Wifi (Xposed Module) - Apps on Google Play, acessado em janeiro 26, 2026, https://play.google.com/store/apps/details?id=eu.chylek.adam.fakewifi&hl=en_US
39. LSPosed Module Brings Back the WiFi & Mobile Data Quick Settings Tiles - YouTube, acessado em janeiro 26, 2026, https://www.youtube.com/watch?v=7o_fndU8fqA
40. How to Fake Battery Percentage Level on Android! - YouTube, acessado em janeiro 26, 2026, <https://www.youtube.com/watch?v=amXef7m6ETM>

41. Running a unrooted Image · Issue #679 · remote-android/redroid-doc - GitHub, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/679>
42. yzygitzh/ReDroid: A toolbox for detecting and countering anti-sandbox behaviors in Android apps - GitHub, acessado em janeiro 26, 2026,
<https://github.com/yzygitzh/ReDroid>
43. ReDroid Usage Example | Ziyue Yang's Homepage, acessado em janeiro 26, 2026,
<https://yzygitzh.github.io/android/2017/08/29/redroid-usage.html>
44. UI crashing with inconsistent dri device mapping on redroid:11.0.0-latest #422 - GitHub, acessado em janeiro 26, 2026,
<https://github.com/remote-android/redroid-doc/issues/422>
45. I want to change my device fingerprint : r/AndroidQuestions - Reddit, acessado em janeiro 26, 2026,
https://www.reddit.com/r/AndroidQuestions/comments/cst3el/i_want_to_change_my_device_fingerprint/
46. device_redroid/redroid.prop at redroid-11.0.0 · remote-android/device_redroid - GitHub, acessado em janeiro 26, 2026,
https://github.com/remote-android/device_redroid/blob/redroid-11.0.0/redroid.prop
47. [Android] How to Spoof with LSPosed Guide : r/PokemonGoSpoofing - Reddit, acessado em janeiro 26, 2026,
https://www.reddit.com/r/PokemonGoSpoofing/comments/1dsszs/android_how_to_spoof_with_lsposed_guide/
48. How to spoof uname -rs per process - Unix & Linux Stack Exchange, acessado em janeiro 26, 2026,
<https://unix.stackexchange.com/questions/270019/how-to-spoof-uname-rs-per-process>
49. [Discussion] [Curious] Just curious why Momo is reporting this, since even my most root-paranoid apps work fine. And what even is TEE? : r/Magisk - Reddit, acessado em janeiro 26, 2026,
https://www.reddit.com/r/Magisk/comments/usdxfe/discussion_curious_just_curious_why_momo_is/